

ENGINEERING REFERENCE GUIDE

KOTOR Universal UI and Map Patch

*Gold-build reconstruction, coordinate math, and cross-resolution reproduction
method*

AUTHORITY The current swkotor.exe selected and play-tested by the user is the gold target. Its SHA-256 is D8F0EEBF470660FFBB0DBE9D6953774B937F73F92260FA2D3427189D8B7F6ADE. When historical notes disagree with this binary, the binary plus the latest play test wins.

Item	Verified value
Supported clean source	SHA-256 761F9466F456A83909036BAEBB5C43167D722387BE66E54617BA20A8C49E9886; 4,042,752 bytes
Gold executable	SHA-256 D8F0EEBF470660FFBB0DBE9D6953774B937F73F92260FA2D3427189D8B7F6ADE; 4,046,848 bytes
Validated display	3440 x 1440 fullscreen
Gold map canvas	1720 x 720
Gold marker overlay	1478 x 720
Document date	2026-08-28

Purpose and document map

This guide explains exactly what changed between the supported clean executable and the user-tested gold build, why each layer exists, the coordinate systems involved, and how to generate equivalent values for another resolution. It is both a reconstruction record and an implementation specification for the future universal patcher.

- Artifact identity and the evidence hierarchy.
- The investigation path, including failed candidates and what each failure proved.
- The executable changes, grouped by function rather than by raw byte noise.
- The map, marker, hitbox, minimap, centering, and GUI-anchor equations.
- A reproducible resolution-generation algorithm with worked examples.
- Validation gates, compatibility rules, and known unknowns.

IMPORTANT SCOPE The math-backed map logic is resolution-general. Some older ultrawide constants in the gold executable are empirical 3440 x 1440 compatibility edits rather than fully understood formulas. Those are documented as measured inputs, not disguised as universal truths.

1. Artifact identity and truth hierarchy

Use the following order whenever names, old notes, or candidate numbers conflict:

1. SHA-256 and byte-for-byte identity of the current gold snapshot.
2. Latest user play test of that exact hash.
3. Exact disassembly and PE structure of that hash.
4. Experiment logs and candidate filenames.
5. Visual inference or recollection.

The gold file is byte-identical to `swkotor_phase0_icon_candidate_008_62b39b.exe`. An older experiment log states that candidate 008 left HUD duplication in one earlier test, while the later user test selected this exact hash as the working final build. Keep both facts: the current hash is authoritative, and future resolution validation must explicitly retest HUD duplication, map open/close, and module transitions.

2. Layered architecture of the gold build

```
supported clean EXE
-> 3440x1440 mode/UI compatibility constants
-> enlarged full-map canvas and logical centering region
-> isolated marker coordinate wrappers
```

```
-> overlay-local mouse hit-test wrapper
-> HUD minimap instance reset wrapper
-> verified gold EXE
```

The essential architectural decision is separation of coordinate domains. The original game uses the same CUIMap class for the large map and the HUD minimap. Changing shared constructor constants alone therefore leaks the enlarged dimensions into the HUD. The gold build keeps the large global defaults but intercepts one HUD construction path and restores retail dimensions only on that instance.

Layer	Responsibility	Why it exists
Display/UI enablement	Accept 3440x1440 and use matching UI resources/centering behavior.	The retail executable recognizes a small resolution set and assumes 4:3-era geometry.
Map size	Set map canvas to 1720x720 and marker overlay to 1478x720.	The original 512x256 canvas is too small for the ultrawide map screen.
Marker wrappers	Convert 440x256 source coordinates into the enlarged overlay domain.	Otherwise notes and the player arrow remain clustered in the original corner.
Hit-test wrapper	Convert centered screen mouse coordinates back to overlay-local coordinates.	Rendered markers and clickable rectangles otherwise disagree.
HUD minimap reset	Restore 512x256 canvas and 440x256 overlay on one constructed instance.	Prevents the enlarged full map from wrapping/duplicating in the gameplay HUD.

3. How the project reached the gold build

1. **Known size patch:** The pre-existing patch enlarged the map but did not transform marker coordinates or click input.
2. **Domain discovery:** Runtime inspection showed map content uses a 512x256 canvas while markers originate in a 440x256 logical domain.
3. **Broad-transform rejection:** Changing shared world-to-map bounds and float scales moved markers but also enlarged fog cells and damaged the gameplay minimap.
4. **Candidate 002:** Used a 1720-pixel overlay width; horizontal marker positions were over-scaled.
5. **Candidate 003:** Used the 1478-pixel overlay but divided by 512, applying the 440/512 correction twice; X was under-scaled.
6. **Candidate 004:** Paired 1478 with the original 440-unit domain. Visual placement became correct, but markers were not clickable.
7. **Candidate 005:** Added a centered screen-to-overlay mouse translation. Clicking worked, but the clickable region remained vertically offset.
8. **Candidate 006:** Added the measured 14-pixel top inset. Marker visuals and hitboxes aligned.

9. **HUD split experiments:** Because CUIMap is shared, enlarged dimensions wrapped into the HUD minimap. Caller-specific and global-split strategies were tested.
10. **Later experiments:** Dynamic-surface and close/deactivation experiments introduced regressions, including map-close crashes. They were not retained.
11. **Final selection:** The live executable was restored to a stable, user-play-tested hash, frozen as D8F0EEBF..., and used as the exact patcher target.

4. Exact changes in the gold executable

4.1 Existing-byte changes

The source-to-gold delta changes 86 bytes inside the original file and appends one 4096-byte PE section. The table groups contiguous byte edits into meaningful operations. VA values assume image base 0x00400000.

VA / location	Clean value	Gold value	Meaning
0x0040AA64 / 0x0040AA84	640 / 480	3440 / 1440	Screen-centering reference width and height.
0x0040B6C6, 0x0040B6D9	-640 / -480	-3440 / -1440	Signed counterparts used by another centering path.
0x0040BA6B, 0x0040BA82	-640 / -480	-3440 / -1440	Signed counterparts used by a second placement path.
0x005F0C64 / 0x005F0C6B	800 / 600	3440 / 1440	Adds the custom mode to a retail resolution acceptance branch.
0x0068C4E2	1024	3440	Redirects a width-based UI resource branch to the custom width.
0x0068C4F3 / 0x0068C4FA	1280 / 1600	0 / 0	Disables later competing width matches in that resource selector.
0x00755788	3/7 $\approx$ 0.4285715	2/7 $\approx$ 0.2857143	Empirical width-derived vertical/UI scalar used at 0x006A74DE and related paths.
0x006928B3 / 0x006928C3	640 / 480	2750 / 1400	Logical content-region dimensions used by the map centering path.
0x0069405B	32	40	Player-arrow/icon size probe retained in the gold build.
0x0069505C / 0x00695064	512 / 256	1720 / 720	Shared CUIMap render/canvas dimensions.
0x00695082 / 0x0069508A	440 / 256	1478 / 720	Shared marker-overlay dimensions.
0x006946F4	call 0x00578E00	call 0x0086D000	Map note/object conversion redirected to scaling wrapper.
0x00694A39 / 0x00694AAC	call 0x005791B0	call 0x0086D080	Party and player conversions redirected to cached-point wrapper.
0x0075477C	0x00693300	0x0086D100	Overlay hit-test virtual function redirected to mouse-translation wrapper.
0x0062B39B	call 0x00694D50	call 0x0086D130	One CUIMap construction path redirected to the HUD minimap reset wrapper.

4.2 New .kui section

Property	Gold value
Section name	.kui
Virtual address	RVA 0x0046D000; VA 0x0086D000
Raw file offset	0x003DB000
Raw size	0x1000 / 4096 bytes
Virtual size	0x01BD / 445 bytes
Characteristics	0x60000020: code, execute, read
+0x000	World-to-map result scaler
+0x080	Party/player cached-point result scaler
+0x100	Overlay hit-test coordinate translator
+0x130	HUD minimap constructor/reset wrapper

The PE header is updated from four sections to five, `SizeOfCode` and `SizeOfImage` are increased, the checksum is zeroed, and a new section header is written into available header space. Most of the 4096-byte raw section is zero padding; only 445 bytes are live wrapper data.

5. Coordinate systems and the underlying math

5.1 Variables

Symbol	Definition	Gold value
W, H	Output screen/root GUI width and height	3440, 1440
Mw, Mh	Full-map canvas width and height	1720, 720
Ow, Oh	Marker-overlay width and height	1478, 720
Dw, Dh	Original marker coordinate domain	440, 256
Tw, Th	Original map texture/canvas domain	512, 256
x, y	Original marker coordinate returned by retail conversion	module dependent
mx, my	Screen-space mouse coordinate	runtime
I	Measured render-viewport top inset	14 pixels

5.2 Full-map canvas rule

For the 3440x1440 release, the chosen canvas is exactly one half of each screen dimension. This is a design rule of the current build, not an engine requirement. A future generator may instead obtain Mw and Mh from a generated map viewport, provided every dependent value uses the same pair.

```
Mw = round(W * 0.5)
Mh = round(H * 0.5)
gold: Mw = 3440/2 = 1720; Mh = 1440/2 = 720
```

5.3 Marker-overlay rule

The retail map canvas is 512 units wide, but the authored marker domain spans 440 units. The overlay therefore covers 440/512 of the enlarged canvas width. Height uses the complete 256-unit domain. Use integer round-to-nearest, matching the wrapper strategy.

```
Ow = floor((Mw * 440 + 256) / 512)
Oh = Mh
gold: Ow = floor((1720*440 + 256)/512) = 1478; Oh = 720
```

5.4 Marker and player-arrow transformation

The original conversion routines remain authoritative for world-to-map placement and module-authored waypoints. Only successful integer results are rescaled. The added half-divisor implements round-to-nearest for non-negative map coordinates.

```
scaled_x = trunc((x * Ow + Dw/2) / Dw)
scaled_y = trunc((y * Oh + Dh/2) / Dh)
with Dw=440 and Dh=256
equivalently: scaled_x ~= round(x * Ow/440)
               scaled_y ~= round(y * Oh/256)
```

WHY NOT SCALE BY 512? Markers are authored in a 440-wide logical region inside the 512-wide canvas. Using 512 as the divisor after already shrinking the overlay to 440/512 applies the horizontal correction twice and under-scales X. That was candidate 003's failure.

5.5 Hitbox and mouse transformation

Marker controls store rectangles in overlay-local coordinates. Rendering centers the canvas inside the owning CUIMap window, but the original custom hit test receives screen/owner coordinates. The wrapper subtracts the runtime centering inset and the measured 14-pixel vertical viewport inset before tail-jumping to the retail hit test. The equation below follows the actual machine code.

```
center_x = arithmetic_shift_right(W - Mw, 1)
center_y = arithmetic_shift_right(H - Mh, 1)
local_x  = mx - center_x
local_y  = my - (center_y + I)
gold: center_x=860; center_y=360; I=14; subtract (860,374)
```

CORRECTION TO AN EARLY NOTE One earlier prose note wrote `local_y = mouse_y - center_y + 14`. The emitted wrapper actually adds 14 to the inset and then subtracts that total, so the byte-accurate formula is `local_y = mouse_y - (center_y + 14)`.

5.6 HUD minimap reset

The wrapper at 0x0086D130 calls the normal CUIMap constructor, then rewrites only that returned instance and its two child rectangles to retail values. This counters the global 1720x720/1478x720 constructor defaults without changing the full-map wrappers.

Object field/control	Full-map global value	HUD reset value
CUIMap +0x0C width domain	1720	512
CUIMap +0x10 height domain	720	256
CUIMap +0x70 render width	1720	512
CUIMap +0x74 render height	720	256
Child +0x1080 map canvas	1720 x 720	512 x 256
Child +0x0E38 marker overlay	1478 x 720	440 x 256

5.7 Logical map-screen centering constants

The function around 0x006928B2 computes an additional translation from the GUI root dimensions and two embedded logical content dimensions. Let Lw and Lh be those patched constants.

```
delta_x = trunc_toward_zero((root_width - Lw) / 2)
delta_y = trunc_toward_zero((root_height - Lh) / 2)
gold: Lw=2750, Lh=1400 -> delta_x=345, delta_y=20
inverse: Lw = root_width - 2*desired_delta_x
         Lh = root_height - 2*desired_delta_y
```

Treat Lw/Lh as the logical content-region dimensions generated for map.gui, not as the map canvas size. For another resolution, first decide the desired extra translation of the map-screen controls, then use the inverse equations. If the GUI generator already places every child in final absolute screen coordinates and no engine translation is wanted, set Lw=W and Lh=H.

5.8 General GUI anchoring math

GUI resources must be generated by anchor policy. Blindly multiplying every coordinate breaks edge-aligned HUD controls, hit regions, and fixed-size art. For a reference layout W0 x H0 with control rectangle (x0,y0,w0,h0):

```
LEFT:      x' = left_margin
RIGHT:     x' = W - (W0 - x0)
TOP:       y' = top_margin
BOTTOM:    y' = H - (H0 - y0)
CENTER:    x' = round((W - w')/2 + center_offset_x)
UNIFORM:   s = min(W/W0, H/H0); w'=round(w0*s); h'=round(h0*s)
SAFE 4:3:  safe_w = min(W, 4*H/3); safe_left=(W-safe_w)/2
```

The current mipc210x7.gui root is 3440x1440. Examples of explicit policies include the top-right menu cluster (RIGHT+TOP), bottom-left party portraits (LEFT+BOTTOM), bottom-right action cluster (RIGHT+BOTTOM), and centered queue/clear controls (CENTER+BOTTOM). Keep fixed pixel dimensions for controls whose textures are not regenerated; scale dimensions only when matching assets are also produced.

6. Worked 3440x1440 example

Step	Calculation	Result
Canvas	$Mw=W/2$; $Mh=H/2$	1720 x 720
Overlay	$Ow=\text{round}(Mw*440/512)$; $Oh=Mh$	1478 x 720
Visual center inset	$((W-Mw)/2, (H-Mh)/2)$	(860, 360)
Hit-test subtraction	$(\text{center_x}, \text{center_y}+14)$	(860, 374)
Map GUI extra delta	$((3440-2750)/2, (1440-1400)/2)$	(345, 20)
Source midpoint marker	$(220,128) \rightarrow$ round by overlay scales	approximately (739,360) in overlay space

The overlay-space X midpoint is approximately 739 rather than 860 because the 440-wide marker region occupies only 440/512 of the full 1720-wide map canvas. Rendering and the owning control supply the remaining horizontal placement. This is precisely why canvas width, overlay width, and marker-domain width must remain distinct variables.

7. Resolution parameter examples

The following values apply the current half-screen canvas policy. They are generation inputs, not claims of visual validation. Every row still requires the test matrix in section 9.

Resolution	Canvas Mw x Mh	Overlay Ow x Oh	Center x,y	Hit-test Y subtraction
1920x1080	960x540	825x540	480,270	284
2560x1080	1280x540	1100x540	640,270	284
2560x1440	1280x720	1100x720	640,360	374
3440x1440	1720x720	1478x720	860,360	374
3840x1600	1920x800	1650x800	960,400	414
3840x2160	1920x1080	1650x1080	960,540	554
5120x1440	2560x720	2200x720	1280,360	374

8. Reproduction algorithm for a new resolution

1. **Identify the executable:** Hash the source. Never transplant offsets into an unknown build. Confirm signatures and PE metadata first.
2. **Generate GUI resources:** Set every GUI root to W x H and transform each child according to an explicit anchor policy. Regenerate dependent textures before scaling control dimensions.
3. **Choose the map canvas:** Default to $Mw = \text{round}(W/2)$, $Mh = \text{round}(H/2)$ to reproduce this release's policy, or use measured generated viewport dimensions.
4. **Calculate the overlay:** $Ow = \text{floor}((Mw * 440 + 256) / 512)$, $Oh = Mh$.
5. **Calculate map-screen logical dimensions:** Choose desired map-control translation and solve $Lw = W - 2 * \text{delta}_x$, $Lh = H - 2 * \text{delta}_y$.
6. **Patch display constants:** Replace direct W/H and signed -W/-H values, update resolution validation/resource selection, and derive or experimentally validate any aspect scalar.
7. **Parameterize the wrappers:** The coordinate wrappers read Ow/Oh from object fields at runtime, so their arithmetic is reusable. The constructor/reset wrapper immediates must match the chosen full-map and HUD dimensions.
8. **Patch atomically:** Verify every original byte or recognized hash, write a temporary file, verify the target hash, create a backup and manifest, then replace.
9. **Validate before naming gold:** Run the complete matrix, capture screenshots and hashes, and freeze the exact passing executable plus required Override resources.

8.1 Generator pseudocode

```
generate_resolution(W, H, source_exe, gui_reference):
    require recognized_hash(source_exe)
    Mw = round_half_up(W / 2)
    Mh = round_half_up(H / 2)
    Ow = (Mw * 440 + 256) // 512
    Oh = Mh
    center_x = (W - Mw) >> 1
    center_y = (H - Mh) >> 1
    hit_y_inset = center_y + measured_top_inset # currently 14
    gui = transform_controls(gui_reference, W, H, explicit_anchor_rules)
    Lw, Lh = solve_logical_map_region(gui, desired_delta)
    patch_exe(W, H, Mw, Mh, Ow, Oh, Lw, Lh, recognized_signatures)
    verify_output_and_write_manifest()
```

8.2 What must be measured instead of guessed

- The 14-pixel top inset at other UI scales or renderer modes. Keep it configurable until cross-resolution tests prove it invariant.

- The width-derived float at 0x00755788. The gold changes 3/7 to 2/7, but its universal aspect formula has not been proven.
- The logical map-screen region Lw/Lh generated by a new map.gui layout.
- Whether the caller at 0x0062B39B remains the HUD instance on another executable build.
- Texture and control sizes when the asset generator changes pixel density.

9. Required validation matrix

Area	Required checks	Failure signal
Launch	Cold launch, main menu, load save, module transition.	Black screen, exception, unsupported mode.
Full map	Open with M, inspect geometry, close with M, repeat 10 times.	Crash, missing art, stale surface, activation/deactivation failure.
Markers	Player arrow, party, blue objects, yellow notes across map extrema.	Clustering, drift, wrong scale, clipping.
Clicks	Click visible center of notes at top/bottom/left/right and cycle arrows.	Offset hitbox, unclickable marker, wrong note selected.
HUD minimap	Walk through long module including engine-room/extreme coordinates.	Vertical wrap, second tile, cut border, wrong zoom.
Fog/grid	Compare cell scale and explored regions with clean behavior.	Broad transform leaked into shared map math.
HUD	Top menu, party portraits, action queue, tooltips, combat mode.	Anchor drift, tiny portraits, clipped controls.
Restore	Patch, verify, restore, hash source, patch again.	Non-deterministic output or backup mismatch.

GOLD GATE A build is not gold because one screenshot looks correct. It becomes gold only after the exact EXE hash and required Override-resource hashes pass the entire matrix at the target resolution.

10. Patcher logic and safety

KOTOR_UI_Gold_Patcher.exe embeds a 4,695-byte custom patch resource, not a full copyrighted executable. The resource contains 35 replacement chunks totaling 4,182 payload bytes. Of the final EXE delta, 86 bytes modify the original file and 4,096 bytes append the .kui section.

- Accept only the exact clean SHA-256 and length.
- Treat the exact gold hash as already patched.
- Refuse all other inputs without creating a backup or output.
- Create and verify swkotor.exe.kotor-ui-backup before replacement.

- Generate into a temporary file and verify the gold hash before atomic replacement.
- Write a JSON manifest with source hash, target hash, paths, version, state, and UTC timestamp.
- Restore only when the installed file is the known gold and the backup is the known clean source.

11. Known limitations and future work

Issue	Current status	Required future action
Only one source build	Patcher supports SHA-256 761F... only.	Add build-specific signatures and wrappers after legal access and testing.
One visually validated resolution	Math is implemented at 3440x1440.	Generate and play-test at least 16:9, 21:9, 16:10, 32:9, and 4K.
Empirical aspect scalar	0x755788 is 2/7 in gold.	Trace owning functions and derive its semantic formula before universal generation.
Top inset	14 pixels at 3440x1440.	Measure at each UI scale; convert to a runtime-derived value if it varies.
Candidate-008 history conflict	Old log and latest user test disagree.	Retest HUD wrap on multiple modules and capture an automated visual/runtime record.
Ebon Hawk note locations	Some stock module waypoints are outside annotated room centers.	Keep universal scaling faithful; correct content only with an optional module-data patch.
GUI/asset automation	Current resources are resolution-specific.	Implement explicit anchor rules, lossless GUI writing, and generated TGA/TXI assets.

Appendix A. Key engine functions and object fields

Address / field	Role
0x00694D50	CUIMap initialization/constructor target.
0x00692810	Per-frame map rendering and logical centering.
0x00693F60	Icon-container and GUI-object initialization.
0x006943D0	Icon positioning/drawing loop.
0x00578E00	Retail world-to-map conversion.
0x005791B0	Retail cached party/player point conversion.
0x00693300	Original overlay custom hit test.
CUIMap +0x0C / +0x10	Normalization width and height.
CUIMap +0x70 / +0x74	Render width and height.
CUIMap +0xE38	Marker-overlay child control.

Address / field	Role
CUIMap +0x1080	Map-canvas child control.

Appendix B. Evidence and reproducibility sources

- Gold snapshot: ..\swkotor_gold_final_D8F0EEBF.exe
- Supported source: ..\swkotor_nopatch.exe
- Patcher source: app\patcher\KotorGoldPatcher.cs
- Delta generator: tools\generate_gold_delta.py
- Coordinate wrapper builder: tools\build_map_icon_draw_wrapper.py
- HUD caller wrapper builder: tools\build_minimap_split_candidate.py
- Global-split comparison: tools\build_global_split_candidate.py
- Reverse-engineering notes: reverse-engineering\map.md
- Experiments: reverse-engineering\experiments\003-isolated-full-map-wrapper.md and 004-global-dimension-split.md
- Machine-readable confirmed marker record:
patches\map_patch\confirmed_icon_coordinate_patch.json

Appendix C. Formula quick reference

```

Mw = round(W/2); Mh = round(H/2)           # current design policy
Ow = floor((Mw*440 + 256)/512); Oh = Mh    # overlay dimensions
sx = Ow/440; sy = Oh/256                  # marker scales
x' = trunc((x*Ow + 220)/440); y' = trunc((y*Oh + 128)/256)
cx = (W-Mw)>>1; cy = (H-Mh)>>1             # runtime centering
local_mouse = (mx-cx, my-(cy+I)); I=14 in current gold # hit test
delta = ((rootW-Lw)/2, (rootH-Lh)/2)      # GUI map translation
Lw=rootW-2*desired_dx; Lh=rootH-2*desired_dy # inverse centering
right_anchor_x = W-(W0-x0); bottom_anchor_y = H-(H0-y0) # GUI anchors

```

MAINTENANCE RULE Never replace a measured unknown with an elegant-looking formula. Record the measured value, the exact bytes and call path, the hypothesis, and the validation needed to promote that hypothesis into reusable math.